

Лекция 2. Системы управления базами данных (СУБД)

Дисциплина: Методы и средства проектирования баз данных

Составитель: Монастырев Егорий Егорьевич, М-ВТ-25

Якутск, 2026

Связь с предыдущей лекцией и цели сегодняшней

Уже знаем (Лекция 1): данные и модели их представления, предметная область, ER-модель (сущность, атрибут, связь).

Сегодня разбираемся:

1. Чем база данных, СУБД и система базы данных отличаются друг от друга?
2. Какие функции обязана выполнять СУБД и зачем?
3. Из каких программных компонентов состоит СУБД изнутри?
4. Как СУБД обслуживает многих пользователей одновременно?
5. Из каких подязыков состоит SQL и как ими пользоваться на практике?

Зачем это нужно уже сейчас

На ближайшей лабораторной работе вы начнёте писать реальные SQL-команды — сегодняшняя лекция даёт для этого необходимый минимум.

Почему одной базы данных недостаточно

Представим, что данные просто лежат в файлах на диске, без управляющей программы.

- ▶ Кто гарантирует, что два сотрудника не испортят одну и ту же запись, редактируя её одновременно?
- ▶ Кто восстановит данные после сбоя электропитания посреди записи?
- ▶ Кто проверит, что оценка не выставлена несуществующему студенту?
- ▶ Кто ограничит доступ так, чтобы студент не мог править чужие оценки?

Вывод

Сами данные не решают эти задачи. Ими управляет специализированное программное обеспечение — **СУБД**.

База данных

Организованная по **схеме** совокупность взаимосвязанных данных, находящаяся под управлением специализированного ПО, предназначенная для совместного использования многими приложениями и пользователями.

СУБД

Комплекс программных средств, реализующий определение схемы, обработку транзакций, поддержание целостности, разграничение доступа, журналирование и восстановление.

Почему не просто «набор данных» / «программа для БД»

Такие бытовые формулировки не отличают базу данных от таблицы Excel и не объясняют, зачем именно нужна СУБД — целостность, многопользовательский доступ, восстановление.

Система базы данных



Понятие	Что это
База данных СУБД	данные + схема программа управления данными
Система БД	данные + СУБД + прило- жения + люди

На примере «Университета»

Данные о студентах — **БД**; PostgreSQL — **СУБД**; портал деканата (БД + СУБД + приложение + сотрудники) — **система базы данных**.

СУБД решает задачи, которые невозможно надёжно решить средствами отдельных файлов и прикладных программ:

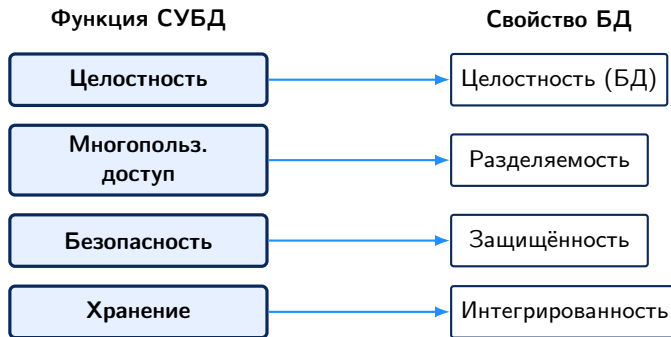
- ▶ предоставляет единый, контролируемый способ доступа к данным;
- ▶ берёт на себя ответственность за корректность данных при сбоях и параллельной работе;
- ▶ скрывает от приложений детали физического хранения данных.

Идея

Приложения работают с данными **через** СУБД, а не напрямую с файлами на диске.

Функция	Суть	Пример («Университет»)
Хранение	таблицы, файлы, страницы	данные о студентах на диске
Обработка запросов	разбор и оптимизация запросов	поиск студента по фамилии
Транзакции	«всё или ничего»	перевод студента в другую группу
Целостность	проверка правил корректности	оценка не привязана к несуществующему студенту
Многопольз. доступ	корректная одновременная работа	два сотрудника правят одну запись
Журналирование и восстановление	защита от потери данных	сбой питания не стирает ведомость
Безопасность	разграничение прав доступа	студент видит только свои оценки

От функций СУБД — к свойствам базы данных



Главная мысль

Свойства базы данных, названные в Лекции 1 (целостность, разделяемость, защищённость, интегрированность), — не сами по себе, а **результат** работы конкретных функций СУБД.

- ▶ **Схема базы данных** — формальное описание структуры БД: таблицы, атрибуты, типы данных, связи, ограничения. Меняется редко.
- ▶ **Экземпляр базы данных** — конкретное состояние данных в определённый момент времени. Меняется практически при каждой операции записи.

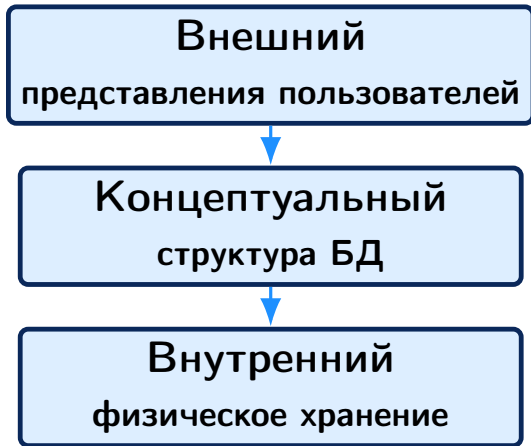
Аналогия для запоминания

Схема — это «тип» (структура), экземпляр — это «значение» (конкретное содержимое) в данный момент.

Пример: схема таблицы `student` (`id`, ФИО, `group_id`) не меняется годами; строки этой таблицы меняются каждый день.

Архитектура ANSI/SPARC и независимость данных

Кратко напомним схему из Лекции 1 и свяжем её с независимостью данных.



Тип независимости	Что позволяет
Физическая	менять хранение и индексы без переписывания приложений
Логическая	менять концептуальную схему, сохраняя внешние представления

Цель всей архитектуры — **независимость данных**.

Важная оговорка

Логическая независимость имеет пределы: удаление атрибута, уже использованного

Важное разграничение

Архитектура ANSI/SPARC отвечает на вопрос «как данные видны снаружи».

Внутренняя архитектура СУБД отвечает на вопрос «из каких программных модулей СУБД состоит изнутри».

Это разные по природе понятия, которые не стоит путать: первое — про уровни абстракции представления данных, второе — про устройство программного ядра.

Компонент	Что делает
Обработчик запросов Менеджер транзакций	разбор и оптимизация запросов контроль атомарности и изоляции операций
Менеджер хранения и буфера	управление файлами, страницами, кэшем в ОЗУ
Менеджер журнала и восстановления	фиксация изменений, восстановление после сбоев
Словарь данных / системный каталог	хранение метаданных о структуре БД

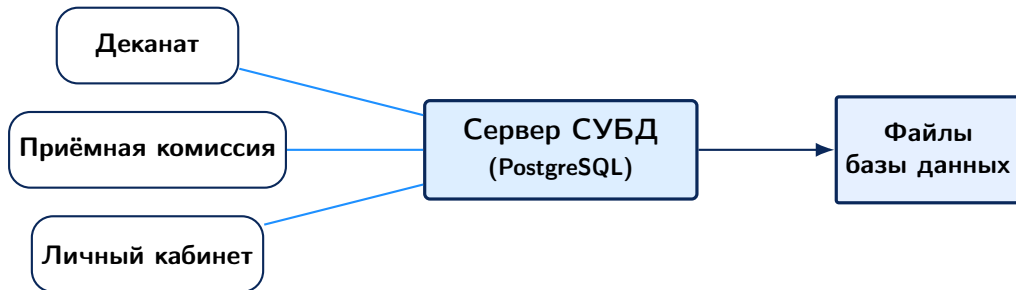
Каждый компонент отвечает за одну из функций СУБД, рассмотренных в разделе о назначении СУБД.

Путь одного запроса через компоненты СУБД



Схема иллюстрирует общий путь запроса без разбора алгоритмов оптимизации: обработчик обращается к словарю данных, чтобы узнать структуру таблиц; менеджер транзакций взаимодействует с менеджером журнала для гарантии устойчивости изменений.

Клиент-серверная модель доступа к СУБД



Идея

Множество клиентских приложений одновременно подключаются к **одному** серверу СУБД, а не к файлам напрямую.

Сотрудник А редактирует оценку студента 101

Сотрудник Б **одновременно** редактирует ту же оценку

- ▶ Что произойдёт, если оба сохраняют изменения одновременно?
- ▶ Как СУБД не допускает потери одного из изменений?

На этом этапе — только постановка вопроса

Механика решения (блокировки, уровни изоляции) рассматривается в отдельной будущей лекции о транзакциях и параллельном выполнении. Сейчас важно зафиксировать: СУБД обязана это контролировать.

Роли участников работы с СУБД

Роль	Что делает
Администратор БД (АБД)	администрирует безопасность, резервное копирование и восстановление, следит за производительностью
Прикладные программисты Конечные пользователи	используют транзакции и целостность через приложение, не работая с ядром СУБД напрямую взаимодействуют с данными через приложение, не видя внутреннего устройства СУБД

В курсе используется **PostgreSQL** — серверная СУБД (в отличие от настольных СУБД вроде Access, уже упомянутых в Лекции 1: ограниченный многопользовательский доступ, локальное использование).

Подключение и работа с сервером PostgreSQL — через отдельные клиентские инструменты:

- ▶ **pgAdmin** — графический интерфейс: таблицы, запросы, права доступа;
- ▶ **psql** — консольный (CLI) клиент для прямой работы с сервером.

На лабораторных работах

Именно через psql / pgAdmin вы будете выполнять SQL-команды, которые рассматриваются далее в этой лекции.

Определение

SQL (Structured Query Language) — стандартизированный декларативный язык для определения структуры данных, выполнения операций над данными и управления доступом в реляционных СУБД.

- ▶ декларативный — пользователь описывает, **что** нужно получить, а не **как** именно это будет вычислено;
- ▶ поддерживается практически всеми реляционными СУБД (с диалектными отличиями; далее — синтаксис PostgreSQL).

Подъязык	Назначение	Связь с функцией СУБД
DDL	определение структуры данных	хранение
DML	работа с данными	обработка запросов
DCL	управление доступом	безопасность
TCL	управление транзакциями	транзакции

Главная идея

SQL — не хаотичный набор команд, а язык, структурно повторяющий функции самой СУБД. Далее — каждый подъязык подробнее, с рабочим синтаксисом для лабораторной работы.

DDL (Data Definition Language) — команды, которые определяют и изменяют структуру базы данных: таблицы, столбцы, ограничения.

Команда	Назначение
CREATE	создать объект (таблицу, индекс и т. д.)
ALTER	изменить структуру существующего объекта
DROP	удалить объект

Связь с функциями СУБД

DDL — практический инструмент функции «**хранение данных**»: этими командами задаётся схема, которую использует СУБД.

```
CREATE TABLE student_group (  
    id          SERIAL PRIMARY KEY,  
    name        TEXT NOT NULL  
);
```

```
CREATE TABLE student (  
    id          SERIAL PRIMARY KEY,  
    full_name   TEXT NOT NULL,  
    group_id    INTEGER REFERENCES student_group(id)  
);
```

Ограничения (PRIMARY KEY, REFERENCES) подробно рассматриваются в Лекции 3; здесь важно увидеть, как DDL превращает логическую структуру в реальные таблицы.

DML (Data Manipulation Language) — команды для чтения и изменения содержимого таблиц.

Команда	Назначение
SELECT	выбрать данные, удовлетворяющие условию
INSERT	добавить новую строку
UPDATE	изменить значения в существующих строках
DELETE	удалить строки

Связь с функциями СУБД

DML напрямую задействует **обработчик запросов** (раздел о внутренней архитектуре) и опирается на **целостность**, которую контролирует СУБД.

```
SELECT full_name FROM student  
WHERE group_id = 1;
```

```
INSERT INTO student (full_name, group_id)  
VALUES ('Иванов И. И.', 1);
```

```
UPDATE student SET group_id = 2  
WHERE id = 5;
```

```
DELETE FROM student WHERE id = 7;
```

Это базовый синтаксис для одной таблицы — достаточный минимум для первой лабораторной работы.

DCL (Data Control Language) — команды для разграничения прав доступа к данным.

Команда	Назначение
GRANT	предоставить право на объект (роли, пользователю)
REVOKE	отозвать ранее предоставленное право

```
GRANT SELECT ON student TO dean_office;  
REVOKE UPDATE ON student FROM guest;
```

DCL — практическая реализация функции **безопасность**: студент из примера про безопасность (раздел о функциях СУБД) видит свои оценки именно благодаря правам, выданным через DCL.

TCL (Transaction Control Language) — команды для управления границами транзакции.

Команда	Назначение
BEGIN	начать транзакцию
COMMIT	зафиксировать все изменения транзакции
ROLLBACK	отменить все изменения транзакции

```
BEGIN;  
UPDATE student SET group_id = 2 WHERE id = 5;  
UPDATE student_group SET name = 'ПМИ-25' WHERE id = 2;  
COMMIT;
```

Обе команды UPDATE выполняются как единое целое («всё или ничего») — это и есть **атомарность** транзакции, упомянутая при определении термина.

Предпросмотр: что дальше в лабораторных работах

Сегодня мы увидели базовый синтаксис DDL, DML, DCL, TCL для работы с одной таблицей.

На лабораторных работах и в следующих лекциях появятся:

- ▶ выборка данных сразу из нескольких таблиц — JOIN;
- ▶ агрегатные функции — COUNT, SUM, AVG, GROUP BY;
- ▶ вложенные запросы (подзапросы);
- ▶ индексы для ускорения запросов (лекция о физическом проектировании).

Важно

Синтаксис, показанный сегодня, — рабочий минимум для первой лабораторной работы; более сложные запросы разбираются отдельно.

От логической модели к физическому хранению



Пользователь и приложение работают только с логической структурой; всё, что ниже, скрыто внутренним уровнем архитектуры ANSI/SPARC.

- ▶ **Файл** — единица хранения таблицы или её части на диске;
- ▶ **Страница** — блок фиксированного размера, которым СУБД читает и записывает данные на диск;
- ▶ **Буфер** — область оперативной памяти, где хранятся недавно использованные страницы, чтобы не обращаться к диску повторно.

Важно

Это обзорное представление. Алгоритмы работы буфера и физической организации файлов — предмет отдельной будущей лекции о физическом проектировании.

Поиск студента по фамилии в таблице из миллиона строк:

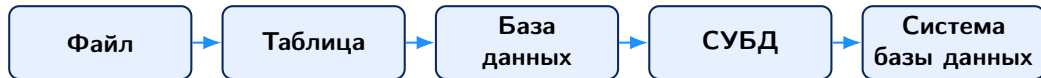
- ▶ **без индекса** — СУБД просматривает все строки подряд;
- ▶ **с индексом** — СУБД использует вспомогательную структуру, которая сразу указывает, где искать нужные строки.

Определение

Индекс — вспомогательная структура данных, ускоряющая поиск записей ценой дополнительного объёма хранения и дополнительных затрат при изменении данных.

Синтаксис создания индекса и стратегии индексирования — предмет отдельной будущей лекции.

Итоговая схема: от файла к системе базы данных



Главная идея лекции

СУБД — не просто программа для хранения таблиц, а комплекс функций (транзакции, целостность, многопользовательский доступ, восстановление, безопасность), реализованных набором программных компонентов и доступных через SQL.

Что будет раскрыто подробнее в следующих лекциях

Тема	Где
Реляционная модель, ключи, ограничения целостности (SQL)	Лекция 3
Нормализация и нормальные формы	Лекция 3 / отдельная лекция
Сложные запросы: JOIN, агрегатные функции, подзапросы	SQL-практикум
Механика транзакций: изоляция, блокировки	Отдельная лекция о транзакциях
Детали журналирования и восстановления	Та же лекция о надёжности
Синтаксис и стратегии индексирования	Лекция о физическом проектировании

1. В чём разница между базой данных, СУБД и системой базы данных?
2. Что различают схема и экземпляр базы данных?
3. Чем архитектура ANSI/SPARC отличается от внутренней архитектуры СУБД?
4. Назовите пять программных компонентов ядра СУБД.
5. Что такое клиент-серверная модель работы с СУБД?
6. Чем отличаются DDL, DML, DCL и TCL — приведите по одной команде на каждый.
7. Какая команда SQL отвечает за атомарное выполнение нескольких операций?
8. Зачем нужен индекс и какова его цена?

Возьмите предметную область и ER-диаграмму, разработанную к Лекции 1.

Напишите:

1. команды `CREATE TABLE` (DDL) для 2–3 сущностей вашей предметной области;
2. по одной команде `INSERT`, `SELECT`, `UPDATE` (DML) на эти таблицы.

Результат: черновой набор DDL/DML-команд — основа для первой лабораторной работы.

Спасибо за внимание!